TECHNICAL DOCUMENTATION · SHARD HEIST 1.1 · TWO ROOMS

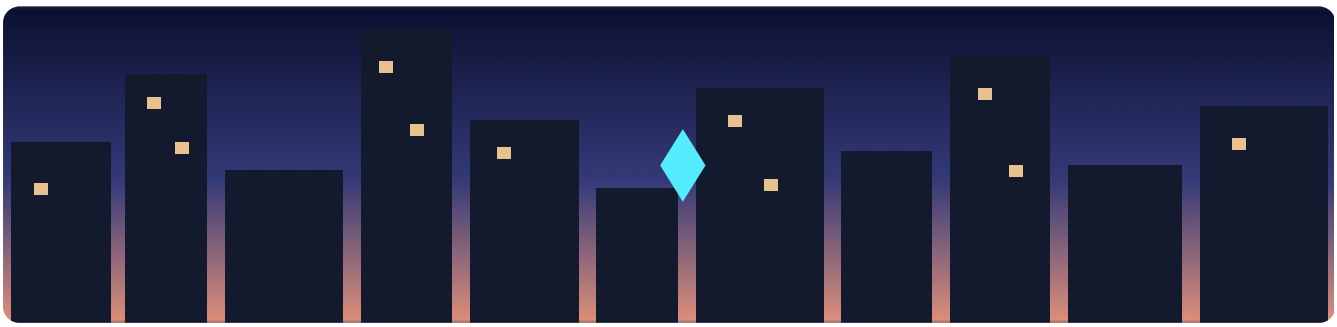
Shard Heist Code Guide

How the two-room browser game is built: the 3D prison escape room (room 1) and the police chase (room 2). For the chase: languages, architecture, the 24 code sections, the per-frame loop, AI and physics, the graphics and audio pipelines, the story mode, testing, security and deployment, an honest complexity assessment and a tuning guide.

Room 1 · Plot Twist: Cell Block D
3D prison escape room: find the code for the hatch in ten minutes (section 14).

Room 2 · Shard Heist
Police chase: three maps, three vehicles, and the story mode The Necklace Job.

01 The game at a glance
02 Languages in the code
03 Complexity assessment
04 Architecture and frame loop
05 Code map: all 24 sections
06 Worlds, traffic and police AI
07 Physics, boundary and camera
08 Graphics and performance
09 Sound, HUD and menu
10 Story mode
11 Tuning guide
12 Limitations and next steps
13 Testing, security and deployment
14 Room 1: the escape room



01 · OVERVIEW

The game at a glance

Shard Heist is a 3D police-chase game that runs entirely in the browser as a static website: one page, one stylesheet and three scripts. The player picks a map (Lisbon's Baixa, Highway or Countryside) and a getaway vehicle (Car, Motorcycle or Truck), collects glowing star shards, escapes up to three police cars and reaches a finish gate before the clock runs out. Crashes cause damage; repair kits remove it; police can bust the player when they box the vehicle in. The story mode **The Necklace Job** is an option in the same main menu (section 10).

5,452	4,947	193	24
LINES OF GAME CODE	LINES OF JAVASCRIPT	NAMED FUNCTIONS	CODE SECTIONS
107	**0**	**0**	**364 KB**
AUTOMATED TESTS	NPM DEPENDENCIES	IMAGE / SOUND FILES	GAME CODE SIZE

Figures for room 2 (index.html, styles.css, src/). Room 1 is described in section 14. The test count covers both rooms: 93 unit and page tests and 14 browser tests. Tests add 1,296 lines.

The three maps

MAP	LAYOUT	GOAL	CAR TOP SPEED	POLICE	TIME
Lisbon · Baixa	6×6 block grid with real street names, traffic lights, 26 AI cars, 3 trams, parked cars	Collect all 8 shards, then reach the gate under the Arco da Rua Augusta	122 km/h	119 km/h	3:00
Highway	4 km, 3 lanes each way, 54 AI vehicles	Reach the finish; 12 shards add +4 s	200 km/h	187 km/h	1:40
Countryside	~3.7 km spline road, hills, farms, village, 18 AI vehicles	Reach the village; 10 shards add +4 s	158 km/h	148 km/h	2:30

What changed since the first edition of this guide (v10)

VERSION	CHANGE
v11	Stutter fix: police light sources no longer change the scene's light count; all shaders are pre-compiled (section 08).
v12	New engine sound built from combustion pulses per cylinder and vehicle; visible map boundary with force field and HUD warning (sections 07, 09).
v13	Audio is suspended when the tab is hidden and silenced in the pause menu.
Edition	The Necklace Job: three-act story mode with an optional hidden letter, a courier lorry and three endings.
1.0	Repository edition: both editions merged into one game; split into index.html, styles.css and src/boot.js, core.js, game.js; game rules moved into core.js with 54 automated tests; error screen; integrity hashes, security policy and validated input; CI with deployment to GitHub Pages (section 13). Two bugs found by the tests were fixed.

1.1	Room 1 added: *Plot Twist: Cell Block D*, a 3D prison escape room in `escape-room/`, separate from the chase until the group connects them (section 14). 24 new tests. Marco's original Python version of Plot Twist sits in `plot-twist/`.
1.2	Engine sound rebuilt as a tested physical model with real engine layouts, per-vehicle gearboxes and separate load and overrun loops (section 09).
1.3	Motorcycle and truck rebuilt: a superbike with a visible inline four, twin-spar frame, upside-down fork, brake discs and an anatomically posed rider; a European cab-over tractor with windscreen, grille, mirrors, open chassis, fuel tank, trailer lines, fifth wheel and twin rear tyres.
1.4	Five wanted stars replace the heat level, so the chase gets harder over time; a police helicopter with a searchlight joins after 60 seconds.
1.5	A real idle sound for each engine; brake lights with a glow on all three vehicles.
1.6	Damage shows in four stages with dents, scratches and loose parts, dark rising smoke and fire; a realistic motorcycle rider who tucks and leans.
1.7	Hills on all three maps: slopes slow the player down or speed them up, and all vehicles ride on them.
1.8	Menu previews rendered from the real 3D models at start-up (`renderPreviews`), with new map pictures.

02 · LANGUAGES

Languages in the code

The game contains **no Python at all**. A web browser can only run JavaScript natively, so all logic is JavaScript. The other languages describe appearance (CSS), page structure (HTML), GPU programs (GLSL) and vector graphics (SVG).



LANGUAGE	LINES	SHARE	CHARACTERS	SHARE	ROLE IN THE GAME
JavaScript	4,947	90.7 %	324,738	87.2 %	All logic: rules (core.js), worlds, vehicles, AI, physics, audio, HUD, menus, error screen
CSS	344	6.3 %	29,715	8.0 %	Look of the HUD, cockpit, menus, warnings, error screen
HTML	138	2.5 %	10,105	2.7 %	Page skeleton, HUD panels, gauge and icons (inline SVG), script order, security policy
GLSL	17	0.3 %	1,554	0.4 %	Sky gradient and the boundary force field (run on the GPU)
SVG in JS strings	6	0.1 %	6,203	1.7 %	Vehicle and map icons in the start menu
Python	**0**	**0 %**	**0**	**0 %**	Not used

Game code of version 1.0: 5,452 lines, 372,315 characters. The tests (1,296 lines), the dev server and SRI checker are JavaScript too; the CI pipeline is YAML. About 170 of the JavaScript lines are comments, including the “###” explanations.

External code that is loaded, not written

LIBRARY	LANGUAGE	PURPOSE
three.js r128	JavaScript (+ GLSL inside)	3D engine: scene, materials, shadows, geometry
three.js examples: EffectComposer, RenderPass, ShaderPass, UnrealBloomPass, CopyShader, LuminosityHighPassShader, GammaCorrectionShader, FXAAShader	JavaScript + GLSL	Post-processing: bloom, gamma, anti-aliasing
Google Fonts	CSS	Bricolage Grotesque and JetBrains Mono typefaces

> **Why no Python?** Python cannot run in a browser page without a server or a large WebAssembly runtime such as Pyodide (several megabytes). A single-file browser game is therefore written in JavaScript. The tests, the dev server and the helper scripts are also JavaScript, run with Node.js. This PDF was rendered from HTML by the Chrome browser, also without Python.

03 · ASSESSMENT

How complex is this code?

Overall: medium to high for a browser game without a framework. In size it sits between a weekend game jam entry (typically 500 to 1,500 lines) and a small indie prototype. What makes it demanding is not the line count but the number of independent systems that cooperate every frame: three procedurally generated worlds, three traffic AIs, a police AI with navigation, arcade physics with collision response, a synthesised engine sound and a post-processing render pipeline.

AREA	≈ LINES	RATING	WHAT MAKES IT HARD
World generation (city, highway, countryside)	770	High	Layout rules, colliders and radar data must agree; batching; height field and spline road
Traffic AI (city, highway, countryside)	300	High	Lane following, intersection arcs, traffic lights, car-following, recycling around the player
Police AI, bust and ramming	240	High	Grid navigation vs. line of sight, stuck recovery, contact physics, heat levels per map
Vehicle modelling	350	Medium	Extruded silhouettes, body shaping, vertex welding, instanced fleets
Procedural textures and surfaces	240	Medium	Canvas painting, tileable noise, normal and roughness maps
Physics, collisions, camera	235	Medium	Multi-circle collisions, impact-based damage, camera kept out of buildings
Engine sound synthesis	175	Medium	Engine model in core.js, load and overrun loops, per-vehicle gearbox, pops, turbo, air brakes
Rendering and graphics pipeline	170	Medium	sRGB/ACES colour, environment map, bloom, MSAA, shader warm-up
Map boundary	80	Low–Med	GLSL force-field shader, distance per map, HUD warning
HUD, radar, cockpit	230	Low–Med	Many DOM values, SVG gauge, rotated canvas radar
Game state, menus, results	280	Low–Med	State machine, cards, stat bars derived from tuning
Story mode	220	Low–Med	Act flow, radio timeline, letter and courier, endings

What helps readability

- Clear section headers with `###` explanations at the top of each of the 24 sections.
- The rules are pure functions in `src/core.js` with automated tests, separate from rendering and audio.
- All tuning lives in a few tables (`MODES`, `VEH` and `ENGINES` in core.js, `STORY` in game.js); the menu stats are computed from them.
- Each map is a “world” object with the same shape: group, colliders, radar shapes.

What makes it harder to maintain

- `game.js` is one closure: its sections can read each other’s variables (for example `car`, `mode`, `police`).
- The 3D, AI and audio code in game.js is only covered by the browser smoke test, not by unit tests.

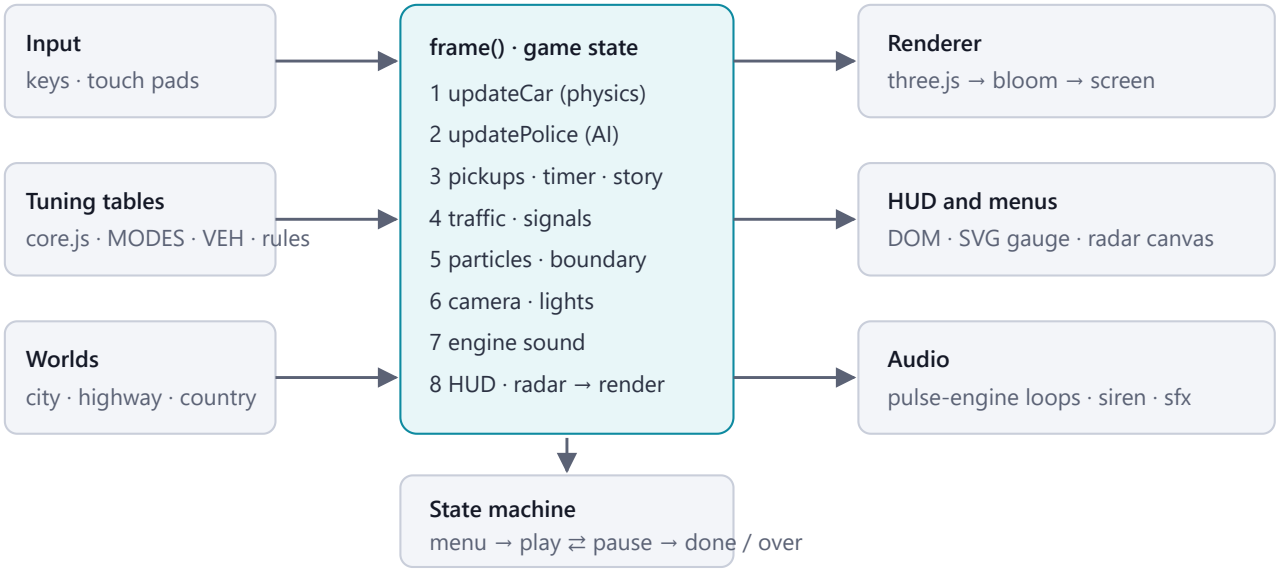
- Mode-specific branches (`if (mode === 'country')`) are spread across physics, police, HUD and pickups.

> **Effort estimate.** A developer who knows three.js needs roughly one to two days to understand the whole code well enough to change it safely; with this guide, about half a day to find their way.

04 · ARCHITECTURE

Architecture and frame loop

The site has five files that load in a fixed order: `index.html` (HUD, menus, error screen), `styles.css`, `src/boot.js` (error screen, loaded first), then three.js, `src/core.js` (the rules as pure functions) and `src/game.js` (one closure with the rest of the game). At start-up the script generates textures and the city, builds the vehicles and police, sets up the renderer, pre-compiles all shaders and shows the menu. The highway and countryside are built only when first selected.



Data flows left to right once per frame: inputs, tuning and world data feed the update steps, which drive rendering, the HUD and audio.

The frame loop

`frame(now)` runs once per display refresh via `requestAnimationFrame`. The time step `dt` (`Core.frameDt`) is never negative and capped at 50 ms, so a slow frame never teleports a vehicle through a wall.

```
// simplified from the Loop section
if (state === 'play') { updateCar(dt); updatePolice(dt); timeLeft -= dt; updatePickups(dt, true);
updateStory(dt); chaseCam(dt); }
else if (state !== 'pause') { /* menu orbits the showroom, results keep the world alive */ }
updateTraffic / updateCountryTraffic / updateSignals + updateNpcs   // per map
updateParticles; updateSmoke; updatePoliceVisuals; updateShowroom; updateBoundary; applyCarMesh
updateEngine; updateHud; drawRadar
composer.render()  or  renderer.render(scene, camera)              // Ultra vs Balanced
```

Game states

STATE	ENTERED BY	WHAT RUNS
`menu`	Page load, “Main menu”, story prologue	Showroom camera orbits the chosen vehicle; city traffic keeps moving; engine silent
`play`	`startRun(map, vehicle)` or `startStoryAct(i)`	Everything: physics, police, timer, pickups, traffic, story timeline

`pause`	P / Esc, tab hidden, window blur	Nothing moves; engine and siren silent; audio suspended while the tab is hidden
`done`	`finish()` at the gate	Result card, or the next story card in story mode
`over`	`gameOver('time' | 'wrecked' | 'busted')`	Failure card, or Ending C in story mode

Every card button carries a `data-action`. `Core.actionAllowed(action, {state, story})` lets it run only in the states whose cards show it, so a stale click or an Enter press on the wrong card does nothing.

05 · CODE MAP

All 24 sections

Line numbers refer to `src/game.js` in version 1.0. Each section starts with a `// ---------- Name ----------` header followed by `// ###` lines that explain it. The tuning tables and the rules (driving, damage, police pressure, scoring, story, input checks) are in `src/core.js`; the error screen is in `src/boot.js`.

LINE	SECTION	PURPOSE	KEY FUNCTIONS
12	Modes & vehicles	Takes the tables and rules from core.js; safe storage access	`Core`, `readStore`, `sel`
33	Renderer & scene	Renderer (with WebGL check), camera, sky, environment map, lights	`tuneEnv`
109	Geometry batching	Merge thousands of boxes into few draw calls	`Batch`, `mergeGeos`
159	Vehicle geometry	Extruded car bodies, instanced fleets	`extrudeProfile`, `shapeBody`, `smoothGeo`, `Fleet`
336	Facade textures	Painted building and shop textures	`makeFacade`, `makeShops`
653	Procedural surfaces	Asphalt, calçada, grass; light cones	`surfaceSet`, `surfaceMat`
753	Worlds	World objects, ground height, traffic lights, city build	`groundH`, `buildCity`, `lightState`
1,444	City traffic	Lane following, turns, red lights, trams on their tracks	`updateNpcs`, `startTurn`, `chooseNext`
1,584	Highway	Motorway, barriers, gantries, scenery	`buildHighway`, `makeSign`
1,711	Highway traffic	Lane traffic both ways, recycling	`updateTraffic`, `spawnVehicle`
1,788	Countryside	Spline road, terrain, farms, village, traffic	`buildCountry`, `roadPoint`, `roadQuery`
2,107	Player vehicles	Coupe, superbike, tractor unit	`setVehicle`
2,547	Police	Spawning, chase AI, ramming, bust meter (rules in core.js)	`updatePolice`, `spawnPolice`, `heat`
2,907	Pickups	Shards, repair kits, finish gate and toll	`placeRun`
2,990	Particles & smoke	Sparks and damage smoke	`burst`, `updateSmoke`
3,069	Sound	Plays the engine model, pops, turbo, air brakes, road, wind, tyres, siren	`setEngineVoice`, `updateEngine`
3,223	Map boundary	Force-field wall, edge distance, HUD warning	`buildBoundary`, `boundaryDistance`, `updateBoundary`
3,299	Input	Keyboard, touch pads, audio suspend, error-screen and lost-graphics hooks	`keys`

3,335	HUD	Gauges, pedals, pursuit, radar	`updateHud`, `drawRadar`, `initGauge`
3,580	Game state	Runs, events, result cards, showroom menu, validated clicks	`startRun`, `finish`, `showMenu`
3,892	Physics	Driving model, collisions, cameras	`updateCar`, `chaseCam`, `camClip`
4,120	Graphics quality	Ultra / Balanced, bloom composer	`applyGfx`, `buildComposer`
4,155	Story mode	Three acts, radio timeline, letter, courier lorry, endings	`startStoryAct`, `updateStory`, `showEnding`, `warmUp`
4,374	Loop	Resize handling and the frame loop	`frame`, `resize`

src/core.js: the rules

GROUP	FUNCTIONS
Tables	`MODES`, `VEH`, `BAIXA`, `streetAt`, `KEY_MAP`, `REV_SPEED`, `REPAIR_AMOUNT`
Stored data and menu input	`parsePick`, `parseBest`, `applyPick`, `actionAllowed`
Driving	`topSpeed`, `stepSpeed`, `collidePoint`, `frameDt`, `vehStats`, `zeroTo80`
Damage and police	`crashDamage`, `addDamage`, `repairDamage`, `isWrecked`, `heatLevel`, `bustStep`, `edgeDistance`
Scoring	`driveTime`, `runStars`, `isNewBest`, `fmtTime`, `fmtTimeExact`
Story	`newStory`, `beginAct`, `retryAct`, `finishOpen`, `storyEnding`, `dueRadio`

Worlds, traffic and police AI

World objects

Every map is represented by the same structure: `{ group, colliders, radar }`. `group` holds all meshes so a whole map can be shown or hidden at once. `colliders` lists boxes (`t:'b'` buildings, `'p'` parked cars, `'w'` walls) and circles (`'c'` trees, poles). `radar` holds rectangles drawn on the mini map. The countryside also keeps a 24 m spatial grid so `collidersNear()` only tests nearby trees out of ~3,000.

City

Lisbon's Baixa as a 6×6 grid of 48 m blocks with 14 m streets. A block is a closed row of Pombaline houses (`block()`): each floor picks its row in the facade texture (`FRONT_ROWS`), so the first floor has the tallest windows and the balconies; a mask (`frontMat`) tints only the plaster, not the limestone. Other blocks are a park or a square. The Arco da Rua Augusta, the National Theatre, Rossio station, the Praça do Comércio towers, the Sé, the river and the bridge are placed by hand. `Core.streetAt` names the street under the car. Lamp posts at block corners double as traffic-light poles.

Highway

A straight 4 km corridor: textured road, dashed lanes, a median barrier and guard rails as long wall colliders, median lights with light cones, sign gantries, forests, hills and a skyline at the finish.

Countryside

A Catmull-Rom spline through 15 seeded control points, sampled every ~2 m. The terrain height field (6 m cells) uses noise hills flattened within 9–75 m of the road, coloured as fields, verges and forest.

Seeded randomness

Layouts use `mulberry32(seed)`, so every player sees the same city and road. Shard positions use `Math.random()` and change every run.

Road coordinates on the countryside

Traffic and police on the country road think in road coordinates: **s** (metres along the road) and **lat** (sideways offset, positive = right of travel). `roadPoint(s, lat)` converts to world `x, z` and heading, interpolating between samples so motion is smooth. `roadQuery(x, z)` does the reverse with a cached search window.

Three traffic systems

MAP	MODEL	BEHAVIOUR
City	Node to node on a grid	Right-hand lane to the next intersection, straight / left / right on a circular arc (3.5 m or 10.5 m radius), stops at red and amber, keeps distance, stops briefly when hit. Leftover distance carries over between straight and arc.
Highway	Fixed lanes, position along z	Lane-dependent cruise speed (fast left lane, lorries right), car-following behind any vehicle including player and police, recycled 380–760 m ahead or behind.
Countryside	Position s along the spline	Both directions in their own lane, same car-following rule, recycled around the player's s.

All three use the same following rule: `desired = min(cruise, speedAhead + (gap − safeGap) × k)`, approached smoothly (proportional change, at most 14 m/s² braking).

Police AI

- **Hills**: `Core.hillHeight` is one height field per map. Each world is built flat, then `laySlopes()` adds vertices to long triangles and bends everything over the field once. Vehicles are placed on it every frame. `Core.slopeTop` and the gravity in `Core.stepSpeed` slow the player uphill and speed them up downhill.
- **Wanted stars**: `Core.wantedStars` gives 1 to 5 stars from the time on the run or the progress, whichever is further along. `Core.PURSUIT` says what each star means: 1 to 4 cars, up to 10 % faster, quicker reinforcements, tougher cars.
- **Helicopter**: joins after 60 seconds, or sooner with the fifth star, and hunts the player with its searchlight. Its searchlight tracks the player at 94 % of police speed (`Core.trackStep`); in the light the police are 6 % faster and see further. The beam is a cone and a ground glow, not a light source, so no shader has to recompile. The rotor sound is a tested loop (`Core.rotorLoop`): four blades at six turns a second make 24 thumps a second, over the tail rotor, turbine and air rumble; it gets louder and brighter as the helicopter nears, with a slight Doppler shift.
- **Navigation**: city cars follow the street grid until `segmentBlocked()` reports a clear line of sight, then drive at the predicted player position. Highway cars stay on their side and swerve around traffic. Country cars follow the road 24 m ahead until within 40 m.
- **Speed control**: slower in sharp turns, faster when far behind, off-road penalty in the countryside, reverse manoeuvre when stuck for 1.2 s.
- **Contact**: the closing speed decides damage to the player (× vehicle `dmg`) and to the police car (× vehicle `ram`). Three “hit points” per police car.
- **Bust**: a police car within 9 m while the player is below ~16 km/h fills the bust meter in 2.2 s; it drains when the player moves away.
- **Light bar**: each car flashes red/blue sprites plus one real point light. The three point lights stay in the scene permanently (see section 08).

Physics, boundary and camera

The driving model is arcade-style and deterministic per frame. It does not simulate tyres or mass; it combines a few simple rules that feel right at speed.

```
// acceleration under throttle (also used by the menu's 0–80 km/h figure)
a = 24 · VEH.accel · (1 − (speed / maxNow)^1.6 · 0.8)        // m/s²
maxNow = MODES.max · VEH.speed · (1 − 0.4 · damage/100) · (off-road ? 0.55 : 1)
// steering: yaw rate falls with speed
turn = steer · min(1, v/6) · 2.5 · VEH.handling / (1 + 0.035 · v)
// damage per crash
damage += max(0, (impact − 3) · 1.6 · VEH.dmg)              // impact = closing speed in m/s
```

Collisions

Each vehicle is approximated by 1–3 circles along its length (`VEH.samples`, radius `VEH.r`). `collidePoint()` tests a circle against a box or circle and returns the push-out normal and depth. Above 2 m/s into the obstacle it counts as a crash (speed drops to 30 %, damage and sparks); below it the vehicle slides along. Traffic and police use the same function with relative velocity.

Map boundary

Every map has an edge: the square around downtown (±147 m), the start and end walls of the highway, and the rim of the countryside terrain. `boundaryDistance()` returns how far the player is from it.

DISTANCE TO EDGE	FEEDBACK
< 46 m	Force-field wall fades in around the approaching spot (GLSL shader: red-gold stripes, scan grid, glowing base line)
< 38 m	Red vignette on the screen edge, growing stronger
< 22 m	“Map boundary · Turn back” banner; its arrow points into the map (to the road on the countryside)
< 1 m (contact)	Force field pulses, 14 sparks, electric zap sound, camera shake; repeats at most every 0.35 s

Ground, slopes and cameras

`groundH(x, z)` returns 0.18 m on city kerbs, 0 on roads and the interpolated height field in the countryside; the vehicle samples four points for pitch and roll. The chase camera follows with speed-dependent distance and field of view, shakes on crashes, is pulled out of buildings by a ray test (`camClip`) and kept above hills. In the menu the showroom camera orbits the vehicle and `setViewOffset` places it beside the menu panel.

Graphics pipeline and performance

STAGE	TECHNIQUE
Colour	All hex colours are sRGB and converted to linear by a small patch on `THREE.Color.setHex`; output uses ACES filmic tone mapping.
Sky and reflections	GLSL gradient shader on a sphere, rendered once into a PMREM environment map so glass and clear-coat paint reflect it.
Draw-call budget	Batching (one mesh per material for a whole map) and instancing (one draw per vehicle part for whole fleets, trees, parked cars).
Materials	PBR: `MeshStandardMaterial`, `MeshPhysicalMaterial` with clear coat for paint, emissive maps for lit windows and signs.
Surfaces	Generated colour, roughness and normal maps: wet asphalt with low-roughness puddles, paving joints, grass.
Shadows	One directional sun with PCF soft shadows; its 120 m shadow box follows the player. 4096^2 on Ultra, 2048^2 on Balanced.
Fake volumetrics	Additive cones under lamps and in front of headlights; glow sprites and points for lamps and vehicle lights.
Post-processing	Ultra: multisampled render target → UnrealBloom → gamma correction (FXAA on WebGL1). Balanced: direct render.

Stutter fix (v11)

three.js compiles a separate shader program for every combination of material and *number of lights*. The police cars originally carried their point light inside the car's group; hiding or showing a car changed the light count, and every material in the scene was recompiled in the middle of a chase.

MOMENT	FRAME TIME BEFORE	AFTER	SHADER PROGRAMS
Normal frame	2–3 ms	1–2 ms	29
1st police car appears	3,576 ms	3 ms	27→43 before, 29 after
2nd police car appears	3,961 ms	1 ms	43→59 before, 29 after

- Police point lights live in the scene permanently and are only switched on and off.
- `warmUp()` compiles every shader at start-up and at each run, temporarily making hidden objects (police cars, gate, smoke, courier) visible to the compiler.
- Smaller savings: one obstacle list per frame instead of one per caller, a stamp-based collider lookup instead of a quadratic duplicate check, and spark materials are freed after use.

Vehicle bodies

A car starts as a 2D side silhouette with arcs for the wheel arches. `extrudeProfile` extrudes it with bevelled edges, `shapeBody` pulls vertices inwards towards the roof (tumblehome) and at the corners, and `smoothGeo` welds vertices so the paint shades smoothly. Glass, lights, lenses and chrome are separate parts with their own materials.

Sound, HUD and menu

Engine sound

The engine is a small **physical model** in `core.js`. `engineLoop()` sends one pressure pulse per cylinder firing into an exhaust pipe that reflects at its open end, then through muffler resonances that stay at the same frequencies at every rpm. The firing frequency is rpm / 60 × cylinders / 2: a 6,000 rpm V8 fires at 400 Hz, a 1,500 rpm truck six at 75 Hz. `engineLoops()` renders each engine at five rpms, once under load and once on overrun, because a real engine sounds different when the driver lifts. The lowest rpm has an idle loop instead: the engine fires at part load with sharp pulses, each cycle a little different, and the valve train, clutch or diesel knock is clearly audible. At runtime its pitch slowly hunts by about 2 %. At runtime `updateEngine()` pitches the loops to the current rpm and crossfades by rpm and throttle, the way racing games blend recorded engine loops.

VEHICLE	ENGINE	RPM RANGE	GEARS	CHARACTER
Car	Cross-plane V8	750–6,800	6	Two banks fire L R R L R L L R into pipes of different length: the V8 burble; overrun pops
Motorcycle	Inline four	1,300–13,000	6	Even firing, short pipe, airbox intake howl; revs fast; 0.07 s shifts; overrun pops
Truck	Inline-six diesel	600–2,300	8	Diesel knock, long pipe, lagging turbo whistle and vent, air brakes; narrow rev band; 0.42 s shifts

- **Signal chain:** loops → compressor. All character is in the loops, so nothing has to be filtered at runtime.
- **Gearbox:** `Core.gearFor` and `Core.engineTarget`, both tested. The throttle closes for each gear change and the revs fall to what the next gear asks for. A rev limiter cuts the fuel at the top.
- **Other layers:** road rumble (stronger off-road), gearbox whine (strongest in reverse), wind rising with speed², tyre squeal, police siren with distance-based volume. A badly damaged engine misfires.
- **Visible damage:** `Core.damageStage` gives four stages. Each model lists its loose parts (`damageParts`), `dentShader` dents and scratches the paint with one uniform, and `Core.smokeFor` sets the smoke from thin grey to a black column, with flames at 88 %.
- **Silence rules:** silent in the menu and pause; all audio is suspended while the tab is hidden.

HUD

ELEMENT	SHOWS	BUILT WITH
Mission panel	Shards, distance to finish, story act title	HTML, inline SVG
Pursuit panel	Active police, chase state, bust meter	HTML, CSS animation
Radar	Buildings or road, traffic, shards, kits, gate, police, courier	Canvas 2D, one world-to-radar transform
Cockpit	Damage, speed arc, real engine rpm, gear P/R/N/D1–6, pedals	SVG generated per map, CSS variables
Boundary warning	Red vignette, banner with direction arrow	HTML, CSS

The HUD only writes to the page when a value changes. In the showroom menu the stat bars are computed by `vehStats()` relative to the best vehicle, and `zeroTo80()` integrates the physics acceleration formula, so the menu always matches the tuning.

10 · STORY MODE EDITION

The Necklace Job

A three-act campaign, started from the main menu, that continues *Plot Twist – The Missing Necklace* and starts where room 1 ends: inmate 4471 has escaped through the tunnel, and Officer Hale, a secret ally, helps over the radio (his lines are orange, police dispatch is red). The detective who investigated the stolen Starfall necklace is framed for the theft; the mastermind is the necklace's owner, Lady Ashford, who staged it for the insurance money. Free ride stays available next to it.

ACT	MAP	GOAL	STORY ELEMENT
1 · The Scatter	City	Recover the 8 shards	Optional sealed letter hidden in a park, not shown on the radar
2 · The Courier	Highway	Catch the gold-marked lorry, take the clasp	The toll at the finish only opens with the clasp
3 · Homecoming	Countryside	Reach the Ashford mansion	Confrontation decides the ending

ENDING	CONDITION	OUTCOME
A · The truth	Letter found	The letter is Lady Ashford's order to frame the detective; she is arrested and the necklace is whole.
B · Framed	No letter	Without proof, the detective goes back to cell 4471.
C · Caught	Busted, wrecked or out of time	Retry the current act; a letter found in an earlier, finished act is kept, one found during the failed attempt is not.

How it is built

- `STORY` holds the acts and a timeline of narrator and police-radio lines per act; `updateStory()` plays them as HUD messages and names the player's vehicle in the radio calls.
- `startStoryAct(i)` reuses `startRun()` for the act's map, then adds the letter (`placeEnvelope`) or the courier lorry (`setupCourier`).
- `finish()` and `gameOver()` hand over to `storyFinish()` and `storyGameOver()` when a story is running; `showEnding()` picks Ending A or B from `story.letter`.
- The rules are in `core.js` and tested in `tests/story.test.js`: `beginAct` (only acts 1–3), `retryAct`, `finishOpen` (the toll needs the clasp), `storyEnding` (A or B) and `dueRadio` (each radio line once).
- The courier lorry drives in the right lane with the same car-following rule as traffic, stops before the toll, and is collidable; traffic treats it as an obstacle.

> **Design rule carried over from the original project:** the letter is optional but decides the ending. Players who only follow the obvious path reach Ending B; players who explore reach the truth.

Tuning guide

Vehicles (`VEH` in src/core.js)

VEHICLE	SPEED	ACCEL	HANDLING	DMG	RAM	COLLISION CIRCLES	MENU BARS (S·A·H·D·R)
Car (muscle coupe)	1.00	1.00	1.00	1.00	1.0	2 × r 1.2 m	4 · 4 · 4 · 3 · 2
Motorcycle (superbike)	1.12	1.30	1.40	1.00	0.4	2 × r 0.8 m	5 · 5 · 5 · 3 · 1
Truck (tractor unit)	0.78	0.70	0.72	0.55	2.4	3 × r 1.4 m	3 · 3 · 3 · 5 · 5

Maps (`MODES` in src/core.js)

MAP	MAX (M/S)	POLICE (M/S)	TIME (S)	SHARDS	KITS	RADAR (M)	BONUS (S)
City	34	33	180	8	3	70	–
Highway	55.6	52	100	12	2	110	4
Countryside	44	41	150	10	3	90	4

Common changes

TO CHANGE…	EDIT
Engine character	`ENGINES` in core.js: cylinders, firing banks, rpm range, gears, shift time, loop rpms, exhaust pipe and muffler resonances, diesel knock, pops, turbo, air brakes, volume
How hard the police chase	`MODES.*.police`, `Core.PURSUIT`, `Core.wantedStars`, `Core.bustStep` (9 m, 4.5 m/s, 2.2 s)
Crash damage	`Core.crashDamage`: factor 1.6, threshold 3 m/s; vehicle `dmg`
Boundary feedback	`updateBoundary()`: 38 m vignette, 22 m banner, 0.35 s zap interval; shader in `boundMat`
Amount of traffic	`N_NPC` (city), `HW_FLEET` (highway), fleet sizes in `buildCountry`
Traffic light timing	`SIGNAL.cycle`, `lightState()` (8 s green, 2 s amber, 1 s all red)
Story texts and timing	`STORY.acts`, `STORY.radio`, card texts in `storyFinish` and `showEnding` (Necklace Job only)
Visual quality	`GFX` presets, bloom in `buildComposer`, `toneMappingExposure`

> After changing anything in `core.js`, run `npm test`. The tests check, among other things, that the menu card still matches the physics and that the bike stays fastest and the truck toughest.

Limitations and next steps

Known limitations

- **Look**: procedural geometry and textures cannot reach photo-realism; vehicles and buildings are stylised.
- **Sound**: the engines are synthesised approximations, not recordings of real engines.
- **Performance**: Ultra (bloom, 4096^2 shadows, 2048^2 facades) needs a decent GPU; phones default to Balanced.
- **AI**: traffic never changes lanes; city police can take detours when the grid route is blocked; police ignore traffic lights.
- **Physics**: no real tyre grip or mass transfer; collisions use circles.
- **Structure**: `game.js` is one closure of about 3,700 lines whose sections share variables.
- **Testing**: the rules and the main browser workflows are tested; long sessions, every map end to end and sound quality are not. CI renders in software.
- **Persistence**: best times live in one browser and can be edited by the player.

Suggested next steps

STEP	BENEFIT	EFFORT
Split the file into ES modules (world, traffic, police, physics, audio, ui, story)	Clear boundaries, easier collaboration	Medium
Give each map a common interface (`build, reset, update, spawnPolice, progress`)	Removes most `if (mode === …)` branches	Medium
Move police steering and traffic following into `core.js` and test them	The AI becomes testable	Medium
Online multiplayer: one player escapes, one drives the police car	Head-to-head chases; a WebSocket server pairs players, relays positions and decides bust and finish with the same core.js rules, so neither client can cheat	High
Online leaderboard	Compare runs between players; needs a backend with login, rate limits and server-side checks of run times	High
Load real glTF vehicle models and recorded engine loops	Large jump in realism	Medium
Lane changes and overtaking for traffic	More natural traffic, better chases	High

> **Where to start reading.** Begin with `src/core.js` and its tests, then `ENGINES` and `frame()` at the very end of `game.js`, then follow `startRun()` into `placeRun()`, `updateCar()` and `updatePolice()`. The `###` comment block at the top of each section tells you what it does before you read the code.

Testing, security and deployment

Repository

PATH	CONTENTS
`escape-room/`	Room 1, self-contained: page, styles, error screen, rules and 3D room (section 14)
`plot-twist/`	The original Plot Twist project in Python/Streamlit (added by Marco, not deployed)
`index.html`, `styles.css`	Room 2: page structure and styling; scripts load in a fixed order
`src/boot.js`	Error screen for load failures, missing WebGL, lost graphics and unexpected errors
`src/core.js`	Tuning tables and all game rules as pure functions
`src/game.js`	3D worlds, vehicles, AI, sound, HUD, story scenes (24 sections)
`tests/`	Node tests for the rules and the page; headless-browser test of the real game
`scripts/`	Local dev server, library integrity checker
`.github/workflows/ci.yml`	Test on every push to main and every pull request; deploy to GitHub Pages only when all tests pass

Tests (78)

FILE	TESTS	ACCEPTED	REJECTED AND EDGE CASES
`rules.test.js`	17	Top speeds, 0–80 times, stat bars, collisions, damage, stars, best times	Gas and brake together, 0 s and 3 s frames, negative frame time, NaN impact, damage outside 0–100, coasting to exactly 0
`story.test.js`	8	Three acts in order, Ending A with the letter, Ending B without	Acts outside 1–3, toll without the clasp, letter found in a failed attempt, radio lines repeating
`input.test.js`	9	Every menu action in its states, valid picks, valid saved data	Actions while driving, story actions without a story, unknown actions, picks like `constructor` or `__proto__`, tampered storage, implausible best times
`page.test.js`	15	Element ids, button actions, script order, dev server serves files; the same page checks for room 1	Unpinned or unhashed libraries, inline scripts, secrets in any file, `.git` requests, path traversal, wrong HTTP methods, one room loading the other's files
`escape-rules.test.js`	15	Both endings, the grace minute, clues, lock, saved games	Actions before the briefing or after an ending, no exit, items twice, tile without spoon, door without mop, wheels that do not exist, clock below zero, tampered saves
`browser.test.js`	14	Room 2: menu, vehicle choice, a run, pause, story act 1. Room 1: briefing, save and continue after reload, Ending A	Card buttons while driving, tampered saved data, blocked library, no WebGL; the tile without the spoon, a wrong code

> **Bugs the tests found (fixed):** saved data such as `{"mode":"constructor"}` passed the old check, because every JavaScript object inherits `constructor`; the next load would have crashed. And the throttle let the car run about 5 km/h above the top speed shown in the menu; it now stops at the limit.

Security

RISK	MEASURE
Credentials or personal data leak	No accounts, no server, no personal data. Only settings and best times in the player's own browser. A test scans the repository for keys and tokens.
Tampered or broken input	Saved data and menu clicks are checked against fixed lists (own properties only); invalid values fall back to defaults; error texts are inserted as text, not HTML.
A CDN serves a changed library	Exact versions plus SRI hashes: the browser refuses any file that differs; `npm run test:sri` re-checks them.
Injected scripts	Content Security Policy: scripts only from this site and the two CDNs, no inline scripts, no plugins, no forms, no foreign base URL.
Vulnerable dependencies	No npm packages; three.js 0.128.0 has 0 known vulnerabilities (npm audit, 3 November 2026); GitHub Actions pinned to commit hashes.
Dev server exposes files	Only known file types inside the project; no hidden files, no `../`, GET and HEAD only.

Deployment

```
git push (main)
  └─ test job:   npm test  →  npm run test:sri  →  npm run test:browser (headless Chrome)
       └─ deploy job (only if all pass):  copy index.html, styles.css, src/, escape-room/  →  GitHub Pages
```

14 · ROOM 1

Plot Twist: Cell Block D

The first room of the game is a 3D escape room that rebuilds the course project *Plot Twist* as a prison break. The detective from the Ashford necklace case has been convicted of the theft and is now **inmate 4471**. Officer Hale locks them in washroom 2 for ten minutes. A hatch in the floor of the toilet stall is held shut by a four-digit lock; the code is hidden in the room. It lives in `escape-room/`, shares no files with the chase, and is played at `/shard-heist/escape-room/`.

BRIEF REQUIREMENT	IN THE ESCAPE ROOM
At least 5 scenes	The door, the sinks, the stall, the hatch, the tunnel
2 items	Spoon (pries the crooked tile off the wall), mop (optional: jams the door)
2 endings	**A**: the hatch opens in time, down into the tunnel. **B**: time runs out, Hale takes you back to the cell
An ending that depends on an earlier decision	Jamming the door with the mop buys one extra minute before Ending B, which can decide whether you reach Ending A

The puzzle

CLUE	WHERE AND HOW	WHAT IT SAYS
Mirror	Only visible after turning on the hot tap: the steam shows letters someone traced	"UNSINKABLE?"
Graffiti	Scratched into the stall wall	"She was unsinkable. She sailed only once. The year opens the floor."
Newspaper scrap	Behind the crooked tile, which only comes off with the spoon	"The 'unsinkable' liner went down on her first voyage. 15 April 19▒2."

Together they point to the Titanic and the code **1912**; "sailed only once" gives the missing digit to players who do not know the history. The code is the same every time. Every wrong code makes noise and costs 30 seconds; Hale warns at 5:00, 2:00, 1:00 and 0:30. The plot twist comes in Ending A: a note on the ladder ("Took you long enough. A car is waiting. — H.") shows the guard wanted the detective out, which leads into the chase.

How it is built

The structure is the one of the original Python project, rewritten in JavaScript: `rules.js` (198 lines) holds the story and the rules as pure functions, `room.js` (776 lines) only draws and turns clicks into actions.

```
// rules.js: one plain object is the whole state
state = { scene, time, inventory, clues, tileOpen, steam, wedged, graceUsed, lock, wrongTries, ending,
warned }
perform(state, action, target)  →  { state, ok, message, cue }   // refused: the SAME state, nothing saved
tick(state, dt)                 →  { state, message, cue }       // the clock, warnings, grace minute,
Ending B
visibleHotspots(state)          →  what can be used right now
restore(savedText)              →  a checked state, or null for anything that is not a real save

// room.js: a click becomes an action
marker or object clicked  →  use(action, target)  →  act()  →  rules.perform()
  accepted: keep the new state, save it, move the camera, syncWorld(), update HUD and markers
  refused:  show the message, play a short buzz, change nothing
```

LINE	SECTION OF ROOM.JS	PURPOSE
12	State and saving	The game state from rules.js, safe storage access, the saved game
27	Renderer, scene, camera	Renderer with WebGL check, shadows, filmic colour, fog
50	Painted textures	Tiles, paint, concrete, diamond plate, graffiti with the clue, steamed mirror, sign, guard silhouette
163	Materials	All surfaces, from glazed tiles to polished steel
190	The washroom	Walls, door and window, sinks and mirror, spoon, mop and bucket, stall, toilet, tile, hatch, shaft, lights
347	Camera	One viewpoint per scene, gliding between them, drag or arrow keys to look around
396	Hotspots	Markers placed over objects, “Walk to” buttons, clicking objects in 3D
456	Actions	use() and act(): everything goes through rules.perform(); syncWorld() moves the objects
494	HUD and cards	Clock, items, clues, messages; briefing, continue, pause, clue, lock and ending cards
648	Sound	Fluorescent hum, drips, running water, effects per action (Web Audio)
702	Endings	Hatch opens and the camera climbs down; or the door bursts open
709	Loop	Clock (rules.tick), animations, markers, render
769	Start	Continue a saved game or start the briefing

Design decisions

- **Markers are real buttons.** Each usable object gets a pulsing HTML button placed over it every frame, so the room works with a mouse, a keyboard and on touch screens, and the browser tests can click them. Clicking the 3D object itself does the same.
- **Viewpoints instead of free walking.** Five fixed spots with free looking around keep the room readable and impossible to get stuck in.
- **The clock keeps running** while reading clues and turning the lock; only the pause card stops it, and so does hiding the tab.
- **Saved after every accepted action**, and checked field by field when loaded, like the SQLite save slot of the original but in the browser.

Limits and next steps

- **Not yet connected** to room 2. Planned: the Ending A card gets a button that opens the chase.
- **Open decisions:** 10 or 15 minutes (`TIME_LIMIT`), and whether the code should change between games (now always 1912).
- Ending B and the grace minute are tested in the rules tests only, not in the browser test.
- The Python original in `plot-twist/` is a separate program with its own pytest tests and is not deployed.